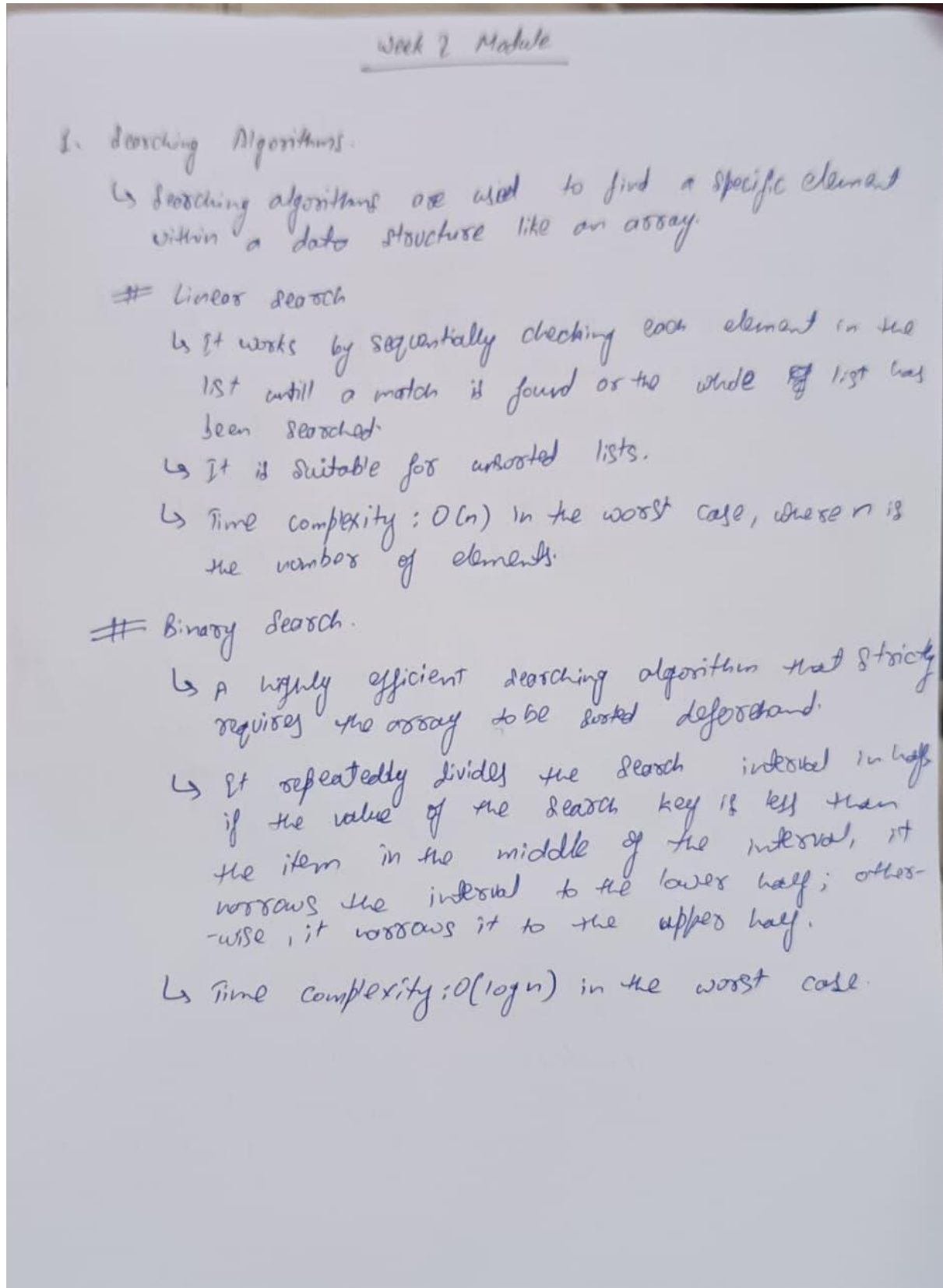


WEEK 2 MODULE

Part 1: Handwritten Notes:



2. Sorting Algorithms.

↳ It rearranges elements in a list according to the specific order (eg, ascending or descending).

Bubble sort

↳ Repeatedly steps through the list, ~~is sorted~~, compares adjacent elements, and swaps them if they are in the wrong order.

↳ The pass through the list is repeated ~~not~~ until the list is sorted, making the largest unsorted element "bubble up" to its correct position.

Insert sort

↳ Builds the final sorted array one item at a time.

↳ It works by taking an element from the unsorted portion and ~~and~~ inserting it into its correct position in the already sorted portion of the array.

Selection sort

↳ Divides the input list into two parts: a sorted sublist and an unsorted sublist.

↳ It repeatedly selects the minimum (or maximum) element from the unsorted sublist and swaps it with the leftmost unsorted element, growing the sorted sublist by one element each time.

Part 2: Programming Problems in C

1. Find Student Roll Number (Linear Search)

```
#include <stdio.h>
int main()
{
    int n, x;
    scanf("%d", &n);
    int roll_numbers[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &roll_numbers[i]);
    }
    scanf("%d", &x);
    int found = 0;
    for(int i = 0; i < n; i++)
    {
        if(roll_numbers[i] == x)
        {
            printf("Found at index %d\n", i);
            found = 1;
            break;
        }
    }
    if(!found)
    {
        printf("Not Found\n");
    }

    return 0;
}
```

Output

5

20 30 45 49 53

53

Found at index 4

=== Code Execution Successful ===

2. Search Product ID (Binary Search)

```
#include <stdio.h>
int main()
{
    int n, target;
    scanf("%d", &n);
    int product_ids[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &product_ids[i]);
    }
    scanf("%d", &target);
    int left = 0, right = n - 1;
    int found = 0;
    while(left <= right)
    {
        int mid = left + (right - left) / 2;
        if(product_ids[mid] == target)
        {
            found = 1;
            break;
        }
        if(product_ids[mid] < target)
        {
            left = mid + 1;
        } else
        {
            right = mid - 1;
        }
    }

    if(found)
    {
        printf("Product Available\n");
    } else
    {
        printf("Product Not Available\n");
    }
    return 0;
}
```

Output

```
7
2001 2002 2003 2004 2005 2006 2007
2003
Product Available

=== Code Execution Successful ===
```

3. Arrange Exam Scores (Bubble Sort)

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int scores[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &scores[i]);
    }
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = 0; j < n - i - 1; j++)
        {
            if(scores[j] > scores[j + 1])
            {
                int temp = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = temp;
            }
        }
    }
    for(int i = 0; i < n; i++)
    {
        printf("%d ", scores[i]);
    }
    printf("\n");
    return 0;
}
```

Output

6

03 99 75 15 21 09

3 9 15 21 75 99

=== Code Execution Successful ===

4. Arrange Library Book Numbers (Insertion Sort)

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int books[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &books[i]);
    }
    for(int i = 1; i < n; i++)
    {
        int key = books[i];
        int j = i - 1;
        while(j >= 0 && books[j] > key)
        {
            books[j + 1] = books[j];
            j = j - 1;
        }
        books[j + 1] = key;
    }
    for(int i = 0; i < n; i++)
    {
        printf("%d ", books[i]);
    }
    printf("\n");
    return 0;
}
```

Output

5

76 49 78 26 14

14 26 49 76 78

=== Code Execution Successful ===

5. Rank Players by Score (Selection Sort)

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int scores[n];
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &scores[i]);
    }
    for(int i = 0; i < n - 1; i++)
    {
        int min_index = i;
        for(int j = i + 1; j < n; j++)
        {
            if(scores[j] < scores[min_index])
            {
                min_index = j;
            }
        }
        int temp = scores[min_index];
        scores[min_index] = scores[i];
        scores[i] = temp;
    }
    for(int i = 0; i < n; i++)
    {
        printf("%d ", scores[i]);
    }
    printf("\n");
    return 0;
}
```

Output

6

10 05 01 48 56 18

1 5 10 18 48 56

=== Code Execution Successful ===

****THE END****